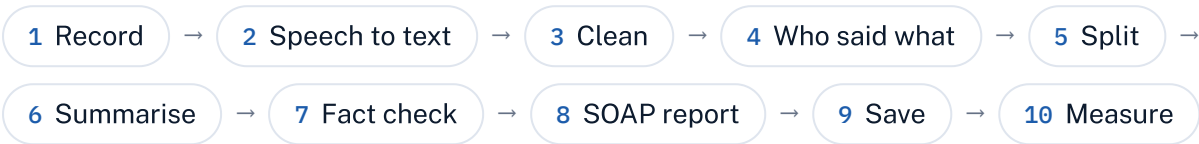


How Clinical Report AI Works

The system listens to a doctor–patient conversation and turns it into a clinical report in the standard SOAP format. This guide follows the conversation through every step, from recording to the saved report. For each step it explains what happens, how it happens, why we chose that method, and what the step did to one real consultation.



The example used throughout is a migraine consultation from the project's test set: nine turns between a doctor and a patient called Robert.



The data

We did not collect patient data and we did not train any model. Every dataset below is used only to test the system.

DATA	WHERE IT CAME FROM	WHAT WE USED IT FOR
Your own consultations	Recorded with the microphone in the web app, or uploaded as an audio file	The real input when the system is used
OSCE practice consultation, 9 min 40 s	Recording of a medical student taking a patient history, already in the project's uploads folder	Testing speech-to-text on real, natural speech
10 benchmark conversations	Written by the authors of the published project, each with a reference summary	Comparing BART with T5, and scoring summaries
MTS-Dialog	Public research dataset of about 1,700 doctor–patient conversations with clinician-written notes (MEDIQA-Chat 2023). The conversations were written by medically trained annotators from sample clinical notes.	Unseen test data: 250 conversations for speaker separation, 53 history-of-illness notes for summaries
Spoken benchmark	The 10 benchmark conversations read aloud by Windows text-to-speech, a male voice for the doctor and a female voice for the patient	Because the true words are known, we can count Whisper's mistakes

WHY NO TRAINING

- Real patient conversations are private and hard to get legally.
- Training large language models needs expensive hardware and a lot of data.

WHY "UNSEEN" DATA MATTERS

- If you test on the same examples you used while building, the score looks better than it really is.
- We never looked at MTS-Dialog while writing our rules, so its scores show how the system does on new conversations.

- Pretrained models already understand language. Our work is choosing the right ones and building a reliable pipeline around them. This is called transfer learning.

1

Record the consultation

The doctor records the conversation in the web app or uploads an audio file.

HOW IT WORKS

1. The doctor opens the web page, which is served by our Python backend (FastAPI).
2. They press **Record** to use the browser's microphone, or upload a file: .wav, .mp3, .m4a, .flac, .ogg or .webm.
3. The browser sends the audio to the server.
4. The server checks the file type and size. Anything over 50 MB or in another format is rejected with a clear message.
5. The file is saved in `data/uploads` with a random 8-character ID added to its name, so two files with the same name never overwrite each other.

Tools: `FastAPI` `Uvicorn` `HTML, CSS, JavaScript`

WHY WE DID IT THIS WAY

- Everything runs on the clinic's own computer. The audio is never sent to an outside service, which protects patient privacy.
- The server only serves files from the uploads folder by their exact name, so a request cannot reach other files on the computer.

2

Speech to text (Whisper)

Whisper listens to the recording and writes down every word, with timings.

HOW IT WORKS

1. FFmpeg, a standard audio tool, converts the file to one format: 16,000 samples per second, one channel.
2. Whisper cuts the audio into 30-second windows.
3. Each window becomes a **spectrogram**: a picture of the sound that shows which pitches are loud at each moment, using 80 pitch bands.
4. The **encoder**, the listening half of the model, reads the picture and turns it into numbers that describe the sounds.
5. The **decoder**, the writing half, writes the text one piece (token) at a time. Each time it picks the most likely next word, given the sound and the words written so far.
6. It also marks where each phrase starts and ends ("segments") and detects the language.
7. The output is the full text, the timed segments and the language. On the GPU, Whisper uses half-precision numbers, which makes it faster.

WHY WHISPER

- Trained by OpenAI on 680,000 hours of speech in many languages, so it copes with accents and background noise.
- Free, open source and runs offline.
- Comes in sizes. Bigger is more accurate but slower: tiny (39 million parameters), base (74 million), small (244 million), medium (769 million). The laptop's GPU lets us run the bigger sizes quickly.
- It can take a short context sentence before listening, such as "Doctor-patient medical consultation about symptoms, vital signs and medications". This steers it towards medical words. We measure whether it helps before turning it on.

ON A REAL RECORDING

The 9 min 40 s OSCE recording with whisper-tiny on the CPU took 2 min 7 s and produced 132 segments and 1,875 words.

Mistakes from the smallest model: "fine-we-year" instead of "final-year", "155 years old" where the patient probably said 55.

This is why we test larger Whisper sizes on the GPU.

Tools: [OpenAI Whisper](#) [FFmpeg](#) [PyTorch on the GPU](#)

3

Clean the text

Remove speech noise and tidy the text without changing any medical fact.

HOW IT WORKS

1. Remove filler words using 15 patterns: "um", "uh", "umm", "hmm", "err", "ah", "like", "you know," (and "you know" at the end of a sentence), "I mean,", "so, yeah", "kind of", "sort of".
2. Remove the gaps and stray commas this leaves behind.
3. Work line by line, so each "Doctor:" or "Patient:" line stays separate.
4. Put one space after a full stop, question mark or exclamation mark, except inside numbers, so 98.6 stays 98.6.
5. Turn repeated punctuation such as "!!" into one mark.
6. Put a space after commas, except inside numbers, so 1,000 stays 1,000.
7. Start every sentence with a capital letter.

WHY IT MATTERS

- The summary models write better summaries from tidy text.
- The published project's cleaning damaged numbers: "98.6" became "98. 6" and "0.5 mg" became "0. 5 mg". In a medical record that is dangerous, so we fixed it.
- It also joined the whole conversation into one line, which stopped the next step from telling the speakers apart. Keeping the lines fixed that too.

IN OUR EXAMPLE

Before

Patient: ... for the past 3 days, mostly on the left side of my head. ~~Um~~, it gets worse with bright light and loud sounds.

"98.6 degrees F", "120/80 mmHg" and "400 mg" in the same conversation come through unchanged.

After

Patient: ... for the past 3 days, mostly on the left side of my head. It gets worse with bright light and loud sounds.

Tools: Python regular expressions (text patterns)

4

Who said what

Label every sentence as Doctor or Patient.

HOW IT WORKS

If the text already has "Doctor:" and "Patient:" labels, as in a typed transcript, the system keeps them. Whisper's text has no labels, so the system works them out:

1. Split the text into sentences.
2. Give each sentence a Doctor score and a Patient score:
 - +1 Doctor for each doctor phrase found. There are 48, such as "what brings you", "let me examine", "I am prescribing", "on a scale", "examination", "we will".
 - +1 Patient for each patient phrase found. There are 28, such as "I have been having", "I feel", "my chest", "I've never", "my wife".
 - +0.5 Doctor for every "you" or "your" (except in "thank you"), and +0.5 Patient for every "I", "me" or "my".
 - +1 Patient when the sentence speaks to the doctor, as in "Thank you, Doctor".
 - Questions: a question about "I" or "we", or about what will happen next ("Will he need an antibiotic?") gets +1 Patient. Any other question ("Did the pain move?") gets +1 Doctor.
3. The higher score wins.
4. If the scores tie and the sentence before was a question, this sentence is the other person's answer. Otherwise the same person is still talking.
5. The first sentence is the doctor's, because consultations start with the doctor's greeting.
6. Neighbouring sentences from the same person are joined into one turn.

WHY RULES

- Fast, needs no extra model or account, and every decision can be explained.
- Measured on 250 unseen public conversations: 88.4% of words go to the right speaker. The published project scored 52.5%, about the same as guessing.
- A future version can also use the voices themselves (speaker diarisation).

A REAL WHISPER SENTENCE PAIR

Whisper output

Did the pain move anywhere else apart from that? No that was it.

After step 4

Doctor: Did the pain move anywhere else apart from that?

Patient: No that was it.

One Whisper segment held both speakers. The question rule and the answer rule split them.

5

Split long conversations

Cut long consultations into overlapping pieces the models can read.

HOW IT WORKS

1. Split the conversation into sentences.
2. Add sentences to a piece until it reaches about 350 words.
3. Start the next piece with the last 40 or so words of the previous one. This overlap keeps the context.
4. If there is only one piece, summarise it once.
5. If there are several, summarise each piece, join those summaries, and summarise them again. This is called hierarchical summarisation.

WHY

- The models can only read a limited amount at once: BART up to 1,024 tokens, T5 up to 512. A token is a word or part of a word.
- Cutting at sentence ends means a sentence is never split in half.

In our example the migraine consultation fits in one piece. The 9-minute OSCE recording became 7 pieces.

6

Write the summary (BART and T5)

An AI model reads the conversation and writes a short summary in its own words.

HOW IT WORKS

1. A **tokenizer** turns the text into tokens, the numbers the model works with. For T5 we put "summarize: " in front, which tells T5 which task to do.
2. The **encoder** reads the whole conversation at once. Through **attention**, every word looks at every other word to understand the context, for example linking "it" back to "the headache".
3. The **decoder** writes the summary one token at a time.
4. **Beam search**: instead of keeping only the single best next word, it keeps the 4 best partial summaries at each step and picks the best finished one.
5. Rules while writing: never repeat the same three words; prefer fuller summaries (length penalty 2.0); make the summary between about a quarter and 60% of the input length, and at most 180 tokens.

- Inputs shorter than 25 words are kept as they are.
- The models run on the GPU when there is one, otherwise on the CPU. Each model loads once, the first time it is needed.

HOW WE CHOSE THE MODELS

- We listed 6 ready-made BART and T5 models from Hugging Face, a public library of pretrained models.
- We ran all 6 on the same 10 benchmark conversations.
- We scored each one with ROUGE and with "facts kept" (see step 10).
- We kept the best BART and the best T5. The app offers both, because the project's objectives ask us to compare them.

MODEL (SUMMARY ONLY, BEFORE FACT CHECK)	TRAINED ON	ROUGE-1	ROUGE-L	FACTS KEPT
BART-large-CNN (published project)	News articles	0.284	0.188	35.8%
BART-large-CNN-SAMSum (chosen BART)	News, then chat conversations	0.369	0.282	69.8%
BART-large dialogue (MEETING_SUMMARY)	Meetings and dialogues	0.321	0.244	62.6%
T5-base (published project)	General web text and tasks	0.280	0.192	39.8%
Flan-T5-base	T5 taught 1,800+ instruction tasks	0.279	0.223	39.9%
Flan-T5-base-SAMSum (chosen T5, best on the benchmark)	Instructions, then chat conversations	0.394	0.312	80.2%

On the 53 unseen MTS-Dialog notes the order changes: BART-large-CNN-SAMSum scores best (ROUGE-L 0.252 against 0.229 for Flan-T5-base-SAMSum). Neither model wins everywhere, so the app keeps both and the doctor can choose.

WHY THESE TWO

- The published project used models trained on news articles, so they copied lines from the conversation instead of summarising it.
- The chosen models were already fine-tuned by other researchers on SAMSum, about 16,000 chat conversations with human-written summaries. They learned to turn a conversation into a short third-person summary.
- Flan-T5 was also taught by Google to follow instructions. It was the most accurate on the benchmark, and it is smaller than BART-large (248 million parameters against 406 million). BART-large-CNN-SAMSum was the most accurate on unseen data.
- We do not train anything. We reuse the published models as they are.

IN OUR EXAMPLE (FLAN-T5-BASE-SAMSUM)

Patient has been having severe throbbing headaches for the past 3 days. He has tried OTC ibuprofen 400 mg twice a day, but it barely helped. The doctor is diagnosing acute migraine headache. He is prescribing Sumatriptan 50 mg and Paracetamol 500 mg for relief. He will follow up in 1 week.

The published project's BART wrote: "Doctor: Good morning, Mr. Robert. How can I help you today? Patient: I have been having...", which is a copy, not a summary.

Tools: Hugging Face Transformers PyTorch

7

Fact check (fact grounding)

Make sure the key numbers and decisions from the conversation are in the summary.

HOW IT WORKS

1. Read these facts from the conversation, never from the AI summary:
 - How long the symptoms have lasted, from the patient's own words.
 - Vital signs and test results, using 10 patterns: blood pressure, heart rate, temperature, breathing rate, oxygen saturation, blood glucose, HbA1c and peak flow.
 - The diagnosis, from doctor phrases such as "I am diagnosing..." or "You have acute..."
 - New medicines with their dose and how often.
 - Timed follow-up, such as "follow up in 1 week".
2. Ignore limits that are not readings, such as "if fever exceeds 102 F".
3. For each fact, check whether the summary already has it: the value of a reading ("120/80"), the name and dose of a medicine, all the main words of a diagnosis.
4. Add only the missing facts at the end, in a short labelled sentence.

WHY

- AI summaries sometimes leave out a dose or a reading. In a medical record that is a real risk.
- Every added fact is copied from the conversation, so nothing is invented.
- Facts kept in the summary rose from 29% in the published project to 89%.
- Trade-off: the extra sentence lowers ROUGE-L slightly for the best model (0.312 to 0.303), because the reference summaries are not written as labelled lists.

IN OUR EXAMPLE

The model's summary already had the diagnosis, both medicines and the follow-up, but it missed the vital signs. The system added:

Findings: Blood Pressure 120/80 mmHg, Temperature 98.6 F.

8

Build the SOAP report

Fill the four sections of a standard clinical note, using rules that copy facts from the conversation.

HOW EACH PART IS FILLED

- **Patient details:** the name after "Mr.", "Mrs.", "Ms." or "my name is"; the age from phrases like "45 years old"; the gender from words like he, she, male, female.
- **S, Subjective (what the patient says)**
 - Chief complaint: the patient's "I have..." or "I've been having..." sentence.
 - Symptoms: matched against a list of 69 symptoms, only in the patient's own sentences. Negatives are skipped: the system looks back up to 6 words for "no", "not", "denies", "without" or "never", so "no chest pain" is not listed.
 - Duration: the first time span the patient mentions, such as "3 days", ignoring ages.
 - Past history: a list of 45 conditions, plus "allergic to ...".
 - Current medicines: drugs the patient says they take, with the dose they say. Questions such as "Do I need an antibiotic?" are ignored.
- **O, Objective (what the doctor finds):** examination sentences ("Examination shows...", "Lungs are clear...") and every reading with its unit.
- **A, Assessment:** the diagnosis and the fact-checked summary.
- **P, Plan:** medicines the doctor prescribes, with dose and frequency. The sentence must contain a prescribing word (prescribe, take, start, increase, add, continue, use) and must not be a question. Then advice (rest, drink, avoid, tests) and follow-up.
- **Recognising medicines:** 107 drug names, plus common drug-name endings (-pril, -sartan, -olol, -statin, -prazole, -cillin, -mycin, -floxacin, -triptan, -gliflozin and more) to catch drugs that are not on the list.
- When something was not said, the report says so, for example "No examination findings recorded". It never fills a gap with a guess.

WHY RULES INSTEAD OF AI HERE

- Doctors must be able to trust every dose. Rules only copy what was said, so they cannot invent a number.
- Each entry can be traced back to a sentence in the conversation, which makes the report easy to check.
- SOAP is the note format doctors already use, so the report fits their workflow.

KNOWN GAPS, NEXT ON OUR LIST

- The symptom list misses everyday words: "nauseous" and "throw up" in our example were not listed.
- "Migraine" was also put under past history, because it is on the history list.
- Small wording glitches, such as the complaint starting "Been having...".

THE REAL REPORT FOR OUR EXAMPLE

S	Subjective	
	Complaint	Been having severe throbbing headaches for the past 3 days, mostly on the left side of my head
	Symptoms	Headache
	Duration	3 days
	History	Migraine
O	Current medicines	Ibuprofen 400 mg twice a day
	Objective	
	Examination	Physical examination is unremarkable
A	Readings	Blood Pressure: 120/80 mmHg · Temperature: 98.6 F
	Assessment	
	Diagnosis	Acute migraine headache
P	Summary	The fact-checked summary from step 7
	Plan	
	Prescribed	Paracetamol 500 mg · Sumatriptan 50 mg
	Advice	Rest in a dark quiet room and drink plenty of fluids · Take at the onset of severe pain
	Follow-up	1 week if the migraine episodes continue

Ibuprofen is listed under current medicines (the patient was already taking it), not under prescribed.

9

Save, search and download

Store each report so it can be found and reopened later.

HOW IT WORKS

1. The doctor reviews the report and clicks **Save**.
2. The server stores one row in a SQLite database: consultation ID, patient ID, doctor, date and time, audio file name, raw transcript, clean transcript, summary, the full SOAP report and the model used.
3. **Search** looks for the typed text in the consultation ID, patient ID, doctor name and transcript.
4. **View** reopens a saved report, **Download** creates a .txt file laid out in SOAP order, and **Delete** removes it.

WHY SQLITE

- The whole database is one file, with no database server to install.
- It is built into Python and is enough for one clinic computer.
- Reports saved before the SOAP change still open, mapped onto the SOAP sections.

Tools: **SQLite** **FastAPI**

Measure accuracy

Score every step with a number, and compare against the published project on the same data.

MEASURE	WHAT IT CHECKS	HOW IT IS CALCULATED
ROUGE-1, ROUGE-2, ROUGE-L	Is the summary close to a human-written one?	ROUGE-1 counts matching single words, ROUGE-2 matching word pairs, ROUGE-L the longest matching run of words in order. Words are cut to their root first ("headaches" becomes "headach"). Score from 0 to 1, averaged over all cases.
Facts kept	Did the summary keep the critical details?	Take every number and drug name in the reference summary, then count how many also appear in our summary.
Speaker accuracy	Did step 4 label the speakers correctly?	Take conversations whose speakers are known, remove the labels, run step 4, and count the share of words given back to the right speaker.
Word error rate	How many words does Whisper get wrong?	Run Whisper on the spoken benchmark, then count the words it swapped, missed or added, divided by the words actually spoken. Lower is better.

KEEPING IT FAIR

- The published project's original code was run unchanged, on the same computer, with the same data and the same scoring scripts.
- MTS-Dialog was never used while building our rules, so its results show performance on new conversations.
- Every result can be repeated with the scripts in the project's `scripts` folder.

RESULTS SO FAR	PUBLISHED PROJECT	OUR PROJECT
Speaker accuracy, 250 unseen conversations	52.5%	88.4%
Summary ROUGE-L, T5, benchmark	0.192	0.303
Facts kept in the summary, T5, benchmark	29.0%	88.9%
Report format	9 custom sections	SOAP
Summary ROUGE-L, BART, 53 unseen MTS-Dialog notes	0.153	0.252
Summary ROUGE-L, T5, 53 unseen MTS-Dialog notes	0.173	0.229
Whisper word error rate	tiny model	being measured

Tools and hardware

TOOL	USED FOR
Python 3.13	The whole backend and all scripts
FastAPI and Uvicorn	The web server and its API (upload, transcribe, summarise, save, search, export, evaluate)
HTML, CSS, JavaScript	The web page: recording, transcript view, report view, database tab, evaluation tab
PyTorch 2.14 with CUDA 13	Runs the AI models on the GPU
Hugging Face Transformers	Downloads and runs the BART and T5 models
OpenAI Whisper and FFmpeg	Speech to text, and audio conversion
SQLite	The report database
rouge-score	The ROUGE measure
Laptop with NVIDIA RTX 4050 (6 GB)	All experiments; the system also runs on a CPU, more slowly

Glossary

Pretrained model

A model already trained by someone else on a large amount of data, which we reuse as it is.

Transfer learning

Using knowledge a model learned on one task for another task.

Encoder and decoder

The encoder reads and understands the input; the decoder writes the output one token at a time.

Parameters

The numbers inside a model that it learned during training. More parameters usually means a more capable but slower model.

Fine-tuning

Extra training of a pretrained model on a smaller, task-specific dataset, such as SAMSum for dialogue summaries.

Transformer

The neural-network design behind Whisper, BART and T5. It uses attention to relate every word to every other word.

Token

A word or part of a word, turned into a number the model can work with.

Beam search

Keeping several candidate sentences while writing and choosing the best finished one.

Spectrogram

A picture of sound showing which pitches are loud at each moment.

SOAP note

The standard clinical note layout: Subjective, Objective, Assessment, Plan.

GPU

A graphics processor. It runs the many small calculations of AI models in parallel, much faster than a CPU.

Word error rate

The share of spoken words that speech-to-text got wrong.

Regular expression

A text pattern used to find or replace words, such as "a number followed by mg".

Held-out (unseen) data

Test data never used while building the system, so the score shows real performance.

ROUGE

A standard score for summaries: how much of a human-written reference summary the generated summary matches.

Clinical Report AI is a documentation aid. It does not diagnose or choose treatment; the doctor reviews every report.